# BetterCode: Reproducible

Someone else, or future you, can rerun everything.

# Overview

- *Recap day 1+2* - readable, reusable
  - Help with code comprehension & abstraction
  - Logical workflow
- **Reproducible**: Environment
  - Help with execution
  - Activity 1: Build a conda environment
  - Activity 2: Turn code to command-line tool

# Reproducible issue in (computational) research

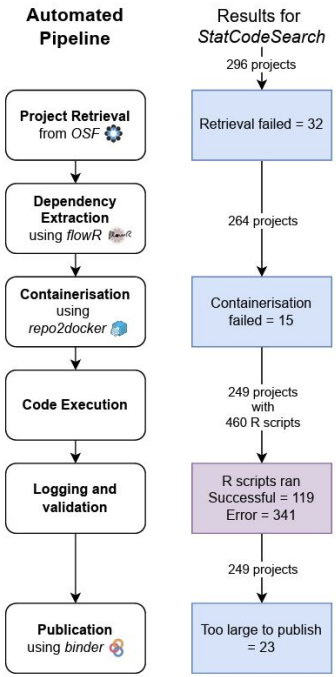



Figure 1: Our completely automated pipeline `osf-to-binder` (left) and the analysis results using this pipeline for the *StatCodeSearch* dataset (right).

*Saju, Lorraine, et al. "Computational Reproducibility of R Code Supplements on OSF." arXiv preprint arXiv:2505.21590 (2025).*

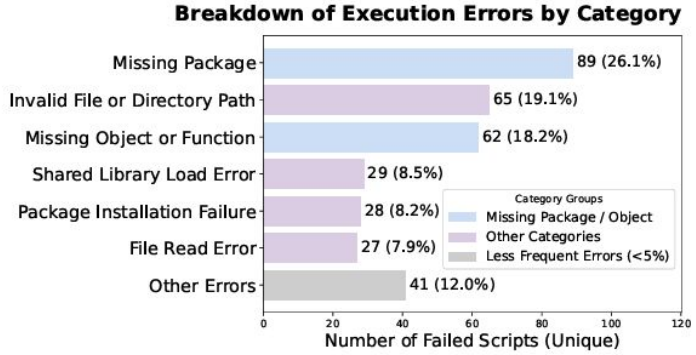
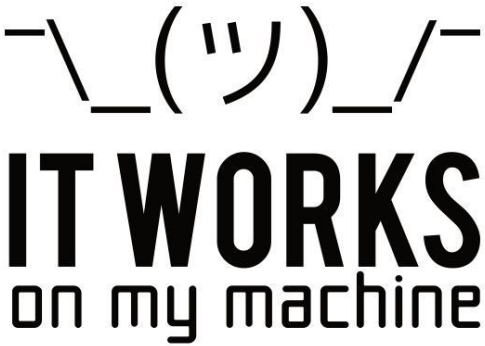



Figure 2: Breakdown of execution errors by category.

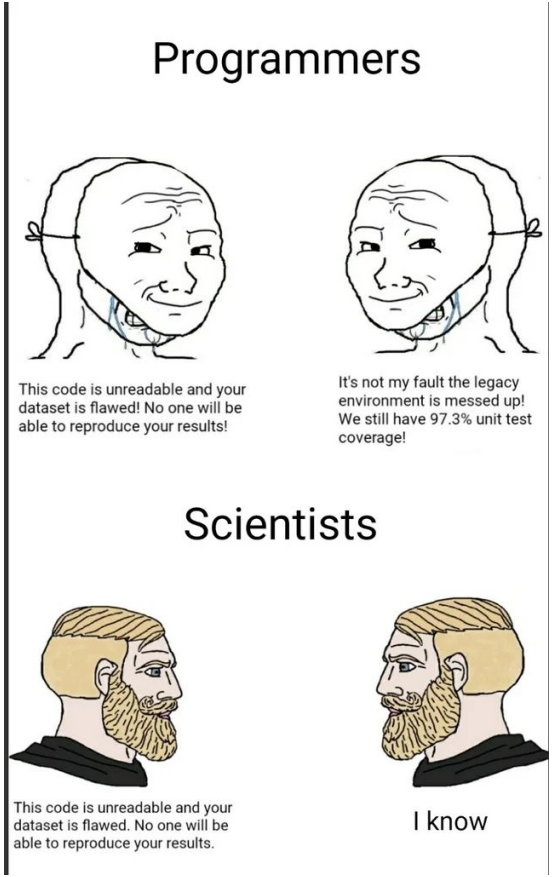
# Why is reproducible important?

- **Foundation** of good science & engineering
- Trust & credibility
- Debugging becomes possible
- Collaboration works
- Saves time, less rework



# What Reproducibility Needs?

- **Defined environment**
- **Executable code that run end-to-end**
- Predictable behaviors
- Proper documentation



# What happens when a code is executed?

| Layer | What it defies | Python example | R example |
|---|---|---|---|
| Code | Intent | *pd.read_csv("data.csv")* | *read_csv("data.csv")* |
| Package/Library | Algorithms + implementation | *pandas::read_csv()* | *readr::read_csv()* |
| **Environment** | Package versions | *pandas == 1.5* | *readr 2.1.4* |
| System libraries | Performance + computation | *libc* (file I/O) | |
| Operating system | Resources | *Linux* file system | |
| Hardware | Physical execution | | |

# Popular options to manage your environment

| Layer | What it defies | Python | R |
|---|---|---|---|
| Code | Intent | (your code) | (your code) |
| Package/Library | Algorithms + implementation | PyPI | CRAN |
| **Environment** | Package versions | venv, **uv** … | **renv** |
| System libraries | Performance + computation | **conda**, mamba, pixi, … | |
| Operating system | Resources | containers (**singularity**, Docker) | |
| Hardware | Physical execution | Physical machine, cloud hardware | |

# Singularity / Apptainer

- How to build a container: resources on MyJAX

https://jacksonlaboratory.sharepoint.com/sites/ResearchIT/SitePages/Containerization-with-Singularity--101.aspx

- Truly reproducible!
- Portable
- Slow iteration, not flexible
- Large image files

# Conda

- How to: bestpractices_workshop:

https://github.com/TheJacksonLaboratory/bestpractices_workshop/blob/main/conda.md

- Flexible
- Mixed-language workflows
- Large disk usage
- Slow dependency resolution

# Language-specific package management

- Quick & (should be) easy
- Project-level isolation
- Language specific
- uv (and other Python-based venv) provides more enviromental control

# My recommendation

In development work:

- conda
- Python-only: uv or other venv management
- R-only: conda + renv

“Established” pipeline: container

# **Demo 1:** Creating environment

Three ways of making setup more explicit:

- `renv` for an R project
- `uv` for a Python project
- `conda` environment

**What should I commit, and what should another person recreate locally?**

# **Activity 1** - Create a Conda Environment

1. Create a conda environment and provide the yaml file.

2. Recreate an environment using a yaml file

# **Demo 2**: Script entrypoint

We have developed and rerun the same analysis pipeline with different ways

- Open an editor, run codes
- Parameterized Notebooks

We now want to make it into a “tool”

- Rscript analysis.R --input data.csv --output results.pdf
- Python analysis.py --input data.csv --output results.pdf

# **Activity 2** - Write an Executable Script

You have a working code to create plots.

Turn it into an executable command-line tool

# Take-away message

- Good reproducibility makes setup, inputs, outputs, and run order visible